PHASE 17 OF 18

SOLID Principles & Design Patterns

SOLID with before/after banking examples, DRY/KISS/YAGNI, creational patterns (Singleton variants, Factory, Builder), structural patterns (Adapter, Decorator, Proxy, Facade), behavioural patterns (Strategy, Observer, Template Method, Chain of Responsibility, Command), patterns in the JDK and in test automation

IN THIS PHASE

17.1 Why Design Principles?
17.2 SOLID Principles
17.3 Design Pattern Categories
17.4 Creational Patterns
17.5 Structural Patterns
17.6 Behavioural Patterns
17.7 Patterns in the JDK and Frameworks
17.8 Patterns in Test Automation (Appium / Selenium)
17.9 Interview Questions
17.10 Quick Revision

17.1 Why Design Principles?

Code that merely works becomes hard to change: one new payment type touches ten classes, a fix in one place breaks another, tests need a real database. **Design principles** keep code **easy to change, test and understand**. **Design patterns** are proven, named solutions to recurring design problems (from the 1994 "Gang of Four" book), so "use a Strategy here" communicates a whole design in three words.

17.2 SOLID Principles

Letter	Principle	One line
S	Single Responsibility	A class should have **one reason to change**
O	Open/Closed	Open for **extension**, closed for **modification**
L	Liskov Substitution	Subtypes must be usable **anywhere** their parent is expected
I	Interface Segregation	Many small, focused interfaces beat one fat interface
D	Dependency Inversion	Depend on **abstractions**, not concrete classes

S: Single Responsibility Principle

```
// BEFORE: one class, four reasons to change (business rules, SQL, PDF layout, SMTP)
class AccountService {
    void transfer(...) { /* rules */ }
    void saveToDb(...) { /* JDBC */ }
    byte[] statementPdf(...) { /* layout */ }
    void emailCustomer(...) { /* SMTP */ }
}

// AFTER: each class has one job and can change or be tested independently
class TransferService { void transfer(...) { } }
class AccountRepository { void save(Account a) { } }
class StatementGenerator { byte[] toPdf(Statement s) { return null; } }
class NotificationService { void email(String to, String msg) { } }
```

O: Open/Closed Principle

```
// BEFORE: every new account type means EDITING this method (risk of breaking others)
double interest(Account a) {
    if (a.type().equals("SAVINGS")) return a.balance() * 0.04;
    else if (a.type().equals("FD")) return a.balance() * 0.07;
    // add RD? NRE? -> keep editing...
    return 0;
}

// AFTER: add a new type by adding a NEW class; existing code stays untouched
interface InterestPolicy { double interest(double balance); }
class SavingsPolicy implements InterestPolicy {
    public double interest(double b) { return b * 0.04; }
}
class FdPolicy implements InterestPolicy {
    public double interest(double b) { return b * 0.07; }
}
class RdPolicy implements InterestPolicy {                  // NEW type: nothing else edited
    public double interest(double b) { return b * 0.065; }
}
```

L: Liskov Substitution Principle

```
class Account {
    void withdraw(double amt) { /* debit */ }
}
class FixedDeposit extends Account {
    @Override void withdraw(double amt) {
        throw new UnsupportedOperationException("FD can't withdraw");   // VIOLATES LSP
    }
}
// Code written for Account ("any account can withdraw") breaks when given an FD.

// FIX: model capabilities, so only withdrawable accounts promise withdraw()
interface Account { double balance(); }
interface Withdrawable extends Account { void withdraw(double amt); }
class SavingsAccount implements Withdrawable { /* ... */ }
class FixedDepositAccount implements Account { /* no withdraw promised */ }
```

> **NOTE**
> Classic LSP smells: a subclass that throws `UnsupportedOperationException`, ignores a method, strengthens preconditions (accepts fewer inputs) or weakens postconditions (guarantees less). The textbook example is `Square extends Rectangle`: setting a square's width also changes its height, breaking code that expects a rectangle.

I: Interface Segregation Principle

```
// BEFORE: every device is forced to implement everything
interface BankingDevice {
    void withdrawCash(); void depositCheque(); void printPassbook(); void scanFingerprint();
}

// AFTER: small role interfaces; classes implement only what they support
interface CashDispenser   { void withdrawCash(); }
interface ChequeDeposit   { void depositCheque(); }
interface PassbookPrinter { void printPassbook(); }
class Atm implements CashDispenser, ChequeDeposit { /* ... */ }
class PassbookKiosk implements PassbookPrinter { /* ... */ }
```

D: Dependency Inversion Principle

```
// BEFORE: high-level policy hard-wired to a low-level detail
class PaymentService {
    private final RazorpayGateway gateway = new RazorpayGateway();   // can't swap, can't mock
}

// AFTER: both depend on an abstraction; the concrete class is INJECTED
interface PaymentGateway { String charge(String acct, double amt); }
class PaymentService {
    private final PaymentGateway gateway;
    PaymentService(PaymentGateway gateway) { this.gateway = gateway; }   // injected
    String pay(String acct, double amt) { return gateway.charge(acct, amt); }
}
new PaymentService(new RazorpayGateway());          // production
new PaymentService((a, amt) -> "MOCK-OK");          // unit test: no network needed
```

> **TIP**
>
> **Dependency Injection** (DI) is the technique; **Inversion of Control** (IoC) is the idea (a container creates and wires objects). Spring's `@Autowired`/constructor injection is DIP in practice. Prefer **constructor injection**: dependencies are explicit, can be `final`, and tests can pass fakes.

Other principles

Principle	Meaning
DRY: Don't Repeat Yourself	One source of truth for each piece of knowledge (validation rules, constants, SQL)
KISS: Keep It Simple	The simplest design that works; avoid cleverness
YAGNI: You Aren't Gonna Need It	Don't build for imagined future requirements
Composition over inheritance	Prefer HAS-A plus interfaces to deep `extends` chains (Phase 6)
Program to an interface	Declare variables, parameters and returns as interfaces (`List`, `PaymentGateway`)
Law of Demeter	Talk to friends, not strangers: avoid `a.getB().getC().doX()` chains
High cohesion, low coupling	Related things together, few dependencies between modules

17.3 Design Pattern Categories

Category	Concern	Patterns covered here
Creational	How objects are created	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural	How classes and objects are composed	Adapter, Decorator, Proxy, Facade, Composite
Behavioural	How objects interact and share responsibility	Strategy, Observer, Template Method, Chain of Responsibility, Command, Iterator, State

17.4 Creational Patterns

Singleton: exactly one instance

Use it for genuinely single shared resources: configuration, a cache, a registry. Overuse creates hidden global state that is hard to test.

```java
// 1. Eager: simple, thread-safe (created at class loading)
public final class ConfigEager {
    private static final ConfigEager INSTANCE = new ConfigEager();
    private ConfigEager() { }
    public static ConfigEager getInstance() { return INSTANCE; }
}

// 2. Lazy with synchronized method: thread-safe but locks on EVERY call
public final class ConfigSync {
    private static ConfigSync instance;
    private ConfigSync() { }
    public static synchronized ConfigSync getInstance() {
        if (instance == null) instance = new ConfigSync();
        return instance;
    }
}

// 3. Double-checked locking: lazy + fast; 'volatile' is REQUIRED
public final class ConfigDcl {
    private static volatile ConfigDcl instance;
    private ConfigDcl() { }
    public static ConfigDcl getInstance() {
        if (instance == null) {                          // 1st check: no lock
            synchronized (ConfigDcl.class) {
                if (instance == null) {                  // 2nd check: with lock
                    instance = new ConfigDcl();
                }
            }
        }
        return instance;
    }
}

// 4. Bill Pugh holder idiom: lazy, thread-safe, no locking (JVM class-init guarantees)
public final class ConfigHolder {
    private ConfigHolder() { }
    private static class Holder { static final ConfigHolder INSTANCE = new ConfigHolder(); }
    public static ConfigHolder getInstance() { return Holder.INSTANCE; }
}

// 5. Enum: simplest; safe against reflection and serialization (Phase 10)
public enum AppConfig { INSTANCE; }
```

Way to break a singleton	Defence
Reflection (`setAccessible` on a private constructor)	Throw from the constructor if an instance exists, or use an **enum**
Serialization (deserializing creates a new object)	Implement `readResolve()` returning `INSTANCE`, or use an enum
Cloning	Don't implement `Cloneable`, or throw `CloneNotSupportedException`
Multiple class loaders	One instance **per class loader** (relevant in app servers)

PITFALL

Without `volatile` in double-checked locking, another thread may see a **partially constructed** object, because instruction reordering can publish the reference before the constructor finishes.

Factory Method / Simple Factory: hide which class gets created

```
interface PaymentProcessor { String process(double amt); }
class UpiProcessor implements PaymentProcessor {
    public String process(double a) { return "UPI " + a; }
}
class CardProcessor implements PaymentProcessor {
    public String process(double a) { return "CARD " + a; }
}
class NetBankingProcessor implements PaymentProcessor {
    public String process(double a) { return "NB " + a; }
}

final class PaymentProcessorFactory {
    private PaymentProcessorFactory() { }
    static PaymentProcessor of(String mode) {
        return switch (mode.toUpperCase()) {
            case "UPI"  -> new UpiProcessor();
            case "CARD" -> new CardProcessor();
            case "NB"   -> new NetBankingProcessor();
            default -> throw new IllegalArgumentException("Unsupported mode: " + mode);
        };
    }
}

PaymentProcessor p = PaymentProcessorFactory.of("upi");   // no 'new UpiProcessor' here
System.out.println(p.process(499));                       // UPI 499.0
```

Callers depend only on the interface, and creation logic lives in one place. JDK examples: `List.of()`, `Calendar.getInstance()`, `NumberFormat.getCurrencyInstance()`, `Executors.newFixedThreadPool()`.

Abstract Factory creates **families** of related objects. For example a `UiFactory` with `AndroidUiFactory`/`IosUiFactory`, each producing a matching `Button` and `TextField`.

Builder: readable construction of complex objects

Use it when a class has **many parameters**, especially optional ones (avoiding "telescoping constructors"). Phase 10 shows a full implementation with a static nested `Builder`.

```
HttpRequest req = HttpRequest.newBuilder()                 // JDK builder (Phase 18)
        .uri(URI.create("https://api.bank.example/v1/accounts"))
        .header("Accept", "application/json")
        .timeout(Duration.ofSeconds(5))
        .GET()
        .build();
```

Other builders: `StringBuilder`, `Stream.builder()`, `Locale.Builder`, and Lombok's `@Builder`.

Prototype

Create new objects by **copying an existing one** (`clone()` or a copy constructor; Phase 8). This is useful when construction is expensive, such as a pre-configured template object.

17.5 Structural Patterns

Adapter: make incompatible interfaces work together

```
interface PaymentGateway { String charge(String acct, double amt); }    // what our code expects

class LegacySoapBankClient {                        // third-party, can't change
    int debitAmountInPaise(String accountNumber, long paise) { return 0; }   // 0 = OK
}

class LegacyBankAdapter implements PaymentGateway {                    // the adapter
    private final LegacySoapBankClient client = new LegacySoapBankClient();
    @Override public String charge(String acct, double amt) {
        int code = client.debitAmountInPaise(acct, Math.round(amt * 100));   // translate
        return code == 0 ? "SUCCESS" : "FAILED:" + code;
    }
}
```

JDK examples: `Arrays.asList()` (array → List), `InputStreamReader` (bytes → chars).

Decorator: add behaviour by wrapping, at runtime

```
interface Notifier { void send(String msg); }
class EmailNotifier implements Notifier {
    public void send(String m) { System.out.println("EMAIL: " + m); }
}

abstract class NotifierDecorator implements Notifier {
    protected final Notifier inner;
    NotifierDecorator(Notifier inner) { this.inner = inner; }
}
class SmsDecorator extends NotifierDecorator {
    SmsDecorator(Notifier n) { super(n); }
    public void send(String m) { inner.send(m); System.out.println("SMS: " + m); }
}
class MaskingDecorator extends NotifierDecorator {
    MaskingDecorator(Notifier n) { super(n); }
    public void send(String m) {
        inner.send(m.replaceAll("\\d(?=\\d{4})", "*"));                 // mask digits
    }
}

Notifier n = new MaskingDecorator(new SmsDecorator(new EmailNotifier()));
n.send("Debited from card 4111222233334444");
// EMAIL: Debited from card ************4444
// SMS: Debited from card ************4444
```

JDK example: the whole `java.io` stack, such as `new BufferedReader(new InputStreamReader(in))` (Phase 12), and `Collections.unmodifiableList(list)`.

Proxy: control access to an object

The proxy has the **same interface** as the real object and adds caching, lazy loading, logging, security checks or remote access. Phase 16 shows a JDK dynamic proxy; Spring's `@Transactional` and `@Cacheable` and Hibernate lazy loading are proxies.

```
class CachedRateProxy implements RateService {
    private final RateService real;
    private final Map<String, Double> cache = new ConcurrentHashMap<>();
    CachedRateProxy(RateService real) { this.real = real; }
    public double rate(String currency) {
        return cache.computeIfAbsent(currency, real::rate);   // hit the slow service once
    }
}
```

Decorator vs Proxy	
Decorator	**Adds** responsibilities; often stacked; the client chooses the combination
Proxy	**Controls access** (caching, security, laziness); usually one; often created by a framework

Facade: one simple entry point to a complex subsystem

```
class FundTransferFacade {
    private final AccountService accounts; private final FraudCheck fraud;
    private final LedgerService ledger;    private final NotificationService notify;
    // constructor omitted
    public String transfer(String from, String to, BigDecimal amt) {
        fraud.verify(from, amt);
        accounts.debit(from, amt);
        accounts.credit(to, amt);
        String ref = ledger.record(from, to, amt);
        notify.send(from, "Transferred " + amt + " ref " + ref);
        return ref;                          // callers make ONE call instead of five
    }
}
```

Composite treats individual objects and groups uniformly through a tree (a `Portfolio` contains `Holding`s and sub-`Portfolio`s, all with `value()`).

17.6 Behavioural Patterns

Strategy: interchangeable algorithms

```
interface FeeStrategy { BigDecimal fee(BigDecimal amount); }

class Payment {
    private final FeeStrategy feeStrategy;
    Payment(FeeStrategy s) { this.feeStrategy = s; }
    BigDecimal total(BigDecimal amt) { return amt.add(feeStrategy.fee(amt)); }
}

FeeStrategy upi  = amt -> BigDecimal.ZERO;                 // lambdas as strategies
FeeStrategy neft = amt -> new BigDecimal("2.50");
FeeStrategy card = amt -> amt.multiply(new BigDecimal("0.018"))
                             .setScale(2, RoundingMode.HALF_UP);

new Payment(card).total(new BigDecimal("1000"));    // 1018.00
new Payment(upi).total(new BigDecimal("1000"));     // 1000
```

A Strategy replaces `if/else` or `switch` on a type with polymorphism. JDK examples: `Comparator` passed to `sort`, `ThreadPoolExecutor` rejection policies.

Observer: notify many listeners of an event

```
interface TxnListener { void onTransaction(String acct, BigDecimal amt); }

class TransactionPublisher {
    private final List<TxnListener> listeners = new CopyOnWriteArrayList<>();
    void subscribe(TxnListener l)   { listeners.add(l); }
    void unsubscribe(TxnListener l) { listeners.remove(l); }      // avoid listener leaks!
    void publish(String acct, BigDecimal amt) {
        listeners.forEach(l -> l.onTransaction(acct, amt));
    }
}

TransactionPublisher pub = new TransactionPublisher();
pub.subscribe((a, amt) -> System.out.println("SMS to " + a + ": debited " + amt));
pub.subscribe((a, amt) -> { if (amt.compareTo(new BigDecimal("50000")) > 0)
                              System.out.println("FRAUD CHECK for " + a); });
pub.publish("SB-1001", new BigDecimal("75000"));
// SMS to SB-1001: debited 75000
// FRAUD CHECK for SB-1001
```

Seen in GUI event listeners, Spring's `ApplicationEventPublisher`/`@EventListener`, Kafka consumers (distributed pub-sub), and the reactive `Flow` API.

Template Method: fixed skeleton, customisable steps

Phase 7's `Loan.emi()` (a `final` algorithm calling the abstract `annualRate()`) is a Template Method. Another example:

```
abstract class StatementJob {
    public final void run() {                // the template: order is fixed
        var data = fetch();
        var doc = format(data);
        deliver(doc);
    }
    protected abstract List<String> fetch();
    protected abstract String format(List<String> rows);
    protected void deliver(String doc) { System.out.println("Emailing statement"); }   // hook
}
```

JDK and frameworks: `AbstractList`, `HttpServlet.service()` → `doGet`/`doPost`, Spring's `JdbcTemplate`, JUnit's lifecycle.

Chain of Responsibility: pass a request along handlers

```
interface TxnValidator { void validate(Txn t); }        // each throws if invalid

List<TxnValidator> chain = List.of(
        t -> { if (!t.kycDone()) throw new IllegalStateException("KYC pending"); },
        t -> { if (t.amount() > 200_000) throw new IllegalStateException("Limit exceeded"); },
        t -> { if (t.isBlacklisted()) throw new IllegalStateException("Fraud risk"); });

chain.forEach(v -> v.validate(txn));    // stops at first failure; easy to add/reorder
```

Seen in servlet filters, Spring Security's filter chain, logging handlers, and exception propagation itself.

Command, Iterator, State (brief)

Pattern	Idea	Example
Command	Wrap a request as an object (queue, log, undo, retry)	`Runnable`/`Callable` submitted to an executor; scheduled jobs; undo stacks
Iterator	Traverse a collection without exposing its internals	`java.util.Iterator`, for-each
State	An object changes behaviour when its internal state changes	Loan status `APPLIED → APPROVED → DISBURSED → CLOSED` modelled as an enum/classes with allowed transitions

17.7 Patterns in the JDK and Frameworks

Pattern	JDK / framework examples
Singleton	`Runtime.getRuntime()`, Spring beans (singleton scope by default)
Factory	`List.of`, `Calendar.getInstance`, `NumberFormat.getInstance`, `Executors.*`, Spring `BeanFactory`
Builder	`StringBuilder`, `HttpRequest.newBuilder()`, `Stream.builder()`, Lombok `@Builder`
Prototype	`Object.clone()`, Spring prototype scope
Adapter	`Arrays.asList`, `InputStreamReader`, Spring MVC `HandlerAdapter`
Decorator	`BufferedInputStream`, `Collections.synchronizedList`/`unmodifiableList`
Proxy	`java.lang.reflect.Proxy`, Spring AOP, Hibernate lazy entities, Mockito mocks
Facade	`javax.faces.context.FacesContext`, Spring `JdbcTemplate` over raw JDBC
Strategy	`Comparator`, `RejectedExecutionHandler`, Spring `Resource` loaders
Observer	`java.util.EventListener`, `Flow.Subscriber`, Spring events
Template Method	`AbstractList`, `HttpServlet`, `JdbcTemplate`, `RestTemplate` callbacks
Chain of Responsibility	Servlet `FilterChain`, Spring Security filters, `java.util.logging` handlers
Iterator	`Iterator`, `Spliterator`

17.8 Patterns in Test Automation (Appium / Selenium)

Pattern	How it appears in a test framework
Page Object Model	One class per screen, holding locators and actions (`LoginPage.login(user, pwd)`); tests read like business steps
Factory	`DriverFactory.create(platform)` returns an `AndroidDriver`, `IOSDriver` or `ChromeDriver` from config
Singleton + ThreadLocal	`DriverManager` holds one driver **per thread** for parallel runs: `ThreadLocal<AppiumDriver>` (call `remove()` after each test)
Builder	Fluent test data: `new CustomerBuilder().withKyc(true).withBalance(5000).build()`
Strategy	Platform-specific gestures or locator strategies chosen at runtime
Decorator / Proxy	`@FindBy` element proxies (PageFactory); wrapping a driver to add logging and screenshots
Facade	`CheckoutFlow` composes several page objects into one business step
Observer	TestNG/JUnit listeners (`ITestListener`, `TestWatcher`) reacting to pass/fail events for reporting

```
public final class DriverManager {
    private static final ThreadLocal<AppiumDriver> DRIVER = new ThreadLocal<>();
    private DriverManager() { }
    public static AppiumDriver get() { return DRIVER.get(); }
    public static void set(AppiumDriver d) { DRIVER.set(d); }
    public static void quit() {
        AppiumDriver d = DRIVER.get();
        if (d != null) { d.quit(); DRIVER.remove(); }   // prevent leaks across pooled threads
    }
}
```

17.9 Interview Questions

Q. Explain SOLID with examples.

S: one reason to change (split transfer logic from persistence and notifications). O: extend through new classes rather than editing switch statements (an InterestPolicy per account type). L: subclasses must honour the parent's contract (an FD shouldn't inherit `withdraw`). I: small role interfaces (CashDispenser vs PassbookPrinter). D: depend on abstractions and inject implementations (a PaymentGateway interface plus constructor injection).

Q. What is the difference between Dependency Inversion, Dependency Injection and IoC?

DIP is the design principle (depend on abstractions). DI is the technique of supplying dependencies from outside (constructor or setter). IoC is the broader idea that a framework controls object creation and the flow (Spring's container).

Q. How do you implement a thread-safe singleton? Which is best?

Eager initialisation, a synchronized accessor, double-checked locking with `volatile`, the Bill Pugh holder idiom, or an enum. An enum is simplest and safe against reflection and serialization; the holder idiom is best for lazy non-enum singletons.

Q. Why is `volatile` needed in double-checked locking?

To prevent reordering that could publish a reference to a not-yet-fully-constructed object to other threads, and to guarantee visibility of the write.

Q. What is the difference between Factory and Builder?

Factory decides **which** class to instantiate, usually in one call. Builder constructs **one complex object step by step**, with many optional parts.

Q. What is the difference between Adapter, Decorator, Proxy and Facade?

Adapter converts one interface to another. Decorator adds behaviour while keeping the interface (stackable). Proxy keeps the interface but controls access (caching, laziness, security). Facade provides a simpler interface over a whole subsystem.

Q. What is the difference between Strategy and Template Method?

Strategy uses **composition**: a whole algorithm is swapped at runtime through an interface or lambda. Template Method uses **inheritance**: a fixed algorithm skeleton in a base class with steps overridden by subclasses.

Q. Where is the Observer pattern used, and what is its risk?

In event listeners, Spring events and pub-sub messaging. The main risk is memory leaks from listeners that are never unsubscribed, plus ordering or exception issues in synchronous notification.

Q. Which design patterns do you use in your automation framework?

Page Object Model, a Factory for drivers, a ThreadLocal-based singleton DriverManager for parallel runs, Builders for test data, Strategy for platform differences, listeners (Observer) for reporting, and PageFactory element proxies.

17.10 Quick Revision

- **SOLID**: SRP (one reason to change), OCP (extend, don't modify), LSP (substitutable subtypes), ISP (small interfaces), DIP (depend on abstractions; inject).
- DRY, KISS, YAGNI, composition over inheritance, program to interfaces, Law of Demeter.
- **Creational**: Singleton (eager / synchronized / DCL + volatile / holder / **enum**), Factory (which class), Abstract Factory (families), Builder (step by step), Prototype (copy).
- Singleton breakers: reflection, serialization (`readResolve`), cloning, multiple class loaders.
- **Structural**: Adapter (convert), Decorator (add by wrapping; java.io), Proxy (control access; Spring AOP), Facade (simplify), Composite (trees).
- **Behavioural**: Strategy (swap algorithm; lambdas), Observer (publish/subscribe), Template Method (final skeleton + hooks), Chain of Responsibility (validators/filters), Command (requests as objects), Iterator, State.
- Automation: POM, DriverFactory, ThreadLocal DriverManager, test-data Builders, listeners.